ctz-euclide

Typst Port

Euclidean Geometry for Typst

A comprehensive geometry package built on CeTZ
Version 0.1.0

Contents

1. Introduction 4
   1.1. Features 4
   1.2. Installation 4
   1.3. Basic Usage 4
2. Core Concepts 5
   2.1. The Point Registry 5
   2.2. Figure Scaling 5
   2.3. Coordinate Systems 5
3. Point Definitions 6
   3.1. Basic Points — `pts` 6
   3.2. Midpoint — `midpoint` 6
   3.3. Regular Polygons — `regular-polygon` 6
   3.4. Linear Combination — `linear` 6
4. Line Constructions 7
   4.1. Perpendicular — `perp` 7
   4.2. Parallel — `para` 7
   4.3. Angle Bisector — `bisect` 7
   4.4. Perpendicular Bisector — `mediator` 8
5. Intersections 9
   5.1. Line–Line — `ll` 9
   5.2. Line–Circle — `lc` 9
   5.3. Circle–Circle — `cc` 9
6. Triangle Centers 11
   6.1. Basic Centers 11
      6.1.1. Centroid — `centroid` 11
      6.1.2. Circumcenter — `circumcenter` 11
      6.1.3. Incenter — `incenter` 11
      6.1.4. Orthocenter — `orthocenter` 12
   6.2. The Euler Line 12
   6.3. Advanced Centers 14
7. Transformations 15
   7.1. Rotation — `rotate` 15
   7.2. Reflection — `reflect` 15
   7.3. Homothety (Scaling) — `scale` 15
   7.4. Projection — `project` 16
8. Drawing & Styling 17
   8.1. Points — `points` 17
   8.2. Labels — `labels` 17
   8.3. Global Styling — `style` 17
   8.4. Angle Marking — `angle` 17
   8.5. Right Angle Mark — `mark-right-angle` 18
9. Circles 19
   9.1. Circumcircle — `circumcircle` 19
   9.2. Incircle — `incircle` 19
   9.3. Circle Through Point — `circle-through` 19
   9.4. Semicircle — `semicircle` 19
10. Clipping 20
   10.1. Set Clip Region — `set-clip` 20
   10.2. Draw Clipped Line — `tkz-line` 20
   10.3. Draw Clipped Segment — `seg` 20
11. Raw Algorithms 21

12. Function Reference 22
12.1. Point Definition 22
12.2. Line Constructions 22
12.3. Intersections 22
12.4. Triangle Centers 22
12.5. Special Triangles 22
12.6. Transformations 22
12.7. Drawing 22
12.8. Clipping 22
13. Gallery Examples 23

1. Introduction

`ctz-euclide` is a geometry package for Typst, a port of the LaTeX package `tkz-euclide`. Built on top of CeTZ (a powerful drawing library), it provides high-level constructions for Euclidean geometry.

1.1. Features

- **Point Registry**: Define points once, reference them by name throughout your figure
- **Geometric Constructions**: Perpendiculars, parallels, bisectors, mediators
- **Intersections**: Line–line, line–circle, circle–circle with multiple solution handling
- **Triangle Centers**: Centroid, circumcenter, incenter, orthocenter, and 10+ specialized centers
- **Special Triangles**: Medial, orthic, intouch triangles
- **Transformations**: Rotation, reflection, translation, homothety, projection
- **Drawing & Styling**: Points, labels, angles, segments with tick marks
- **Grid & Axes**: Coordinate systems with customizable appearance
- **Clipping**: Mathematical line clipping for clean bounded figures

1.2. Installation

Import the package in your Typst document:

```
#import "../src/lib.typ": *
```

All figures use the `canvas` function (re-exported from CeTZ):

```
#canvas({
  import cetz.draw: *
  init()

  // Your geometry code here
})
```

The `init()` call initializes the point registry and coordinate resolver.

1.3. Basic Usage

Code

```
#canvas(length: 0.8cm, {
  import cetz.draw: *
  init()

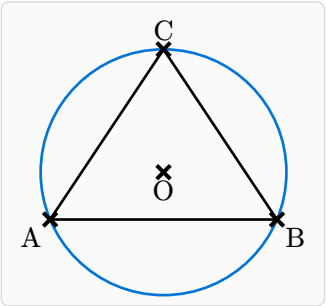
  // Define points
  pts(A: (0, 0), B: (4, 0), C: (2, 3))

  // Draw triangle
  line("A", "B", "C", "A", stroke: black)

  // Find circumcenter and draw circumcircle
  circumcenter("O", "A", "B", "C")
  circle("O", "A", stroke: blue)

  // Draw and label points
  points("A", "B", "C", "O")
  labels("A", "B", "C", "O",
    A: "below left", B: "below right",
    C: "above", O: "below")
})
```

Figure



2. Core Concepts

2.1. The Point Registry

The point registry is the heart of `ctz-euclide`. Once you define a point with a name, that name can be used directly in CeTZ drawing commands.

```
pts(A: (0, 0), B: (3, 4))  // Register points A and B
line("A", "B")              // Use them directly in CeTZ
```

Under the hood, `init()` installs a coordinate resolver that translates `"A"` to the stored coordinates.

2.2. Figure Scaling

Control the size of your figures using CeTZ's `length` parameter:

```
#canvas(length: 0.8cm, { ... })
```

This scales everything proportionally, including stroke widths. Typical values:

- `0.6cm` – small inline figures
- `0.8cm` – standard examples
- `1.0cm` – large detailed figures

2.3. Coordinate Systems

Points can be defined in multiple ways:

```
// Explicit coordinates
pts(A: (2, 3))

// Using existing CeTZ coordinates
pts(B: (rel: (1, 1), to: "A"))

// Mixed: numbers and existing points
pts(C: (4, 0), D: "A", E: (3, 2))
```

3. Point Definitions

3.1. Basic Points — `pts`

Define one or more points at specific coordinates:

```
pts(A: (0, 0), B: (4, 0), C: (2, 3))
```

3.2. Midpoint — `midpoint`

Find the midpoint of a segment:

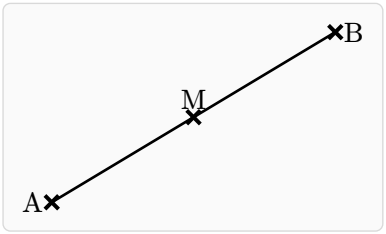
Code

```
#canvas(length: 0.8cm, {
  import cetz.draw: *
  init()

  pts(A: (0, 0), B: (5, 3))
  midpoint("M", "A", "B")

  line("A", "B", stroke: black)
  points("A", "B", "M")
  labels("A", "B", "M",
    A: "left", B: "right", M: "above")
})
```

Figure



3.3. Regular Polygons — `regular-polygon`

Generate vertices of a regular n-gon:

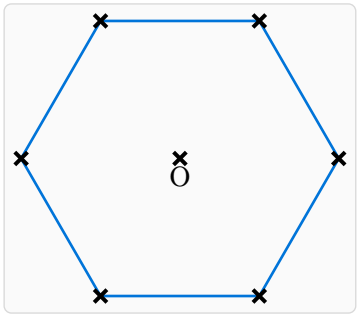
Code

```
#canvas(length: 0.7cm, {
  import cetz.draw: *
  init()

  pts(O: (0, 0), A: (3, 0))
  regular-polygon(("A", "B", "C", "D", "E", "F"),
"O", "A")

  line("A", "B", "C", "D", "E", "F", "A", stroke:
blue)
  points("A", "B", "C", "D", "E", "F", "O")
  labels("O", O: "below")
})
```

Figure



3.4. Linear Combination — `linear`

Define a point along a line: $P = A + k(B - A)$

```
linear("P", "A", "B", 0.3)  // P is 30% from A to B
linear("Q", "A", "B", 1.5)  // Q extends beyond B
```

4. Line Constructions

4.1. Perpendicular — `perp`

Construct a perpendicular line through a point:

Code

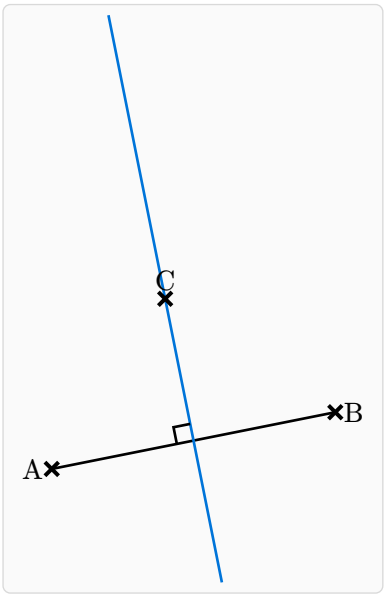
```
#canvas(length: 0.75cm, {
  import cetz.draw: *
  init()

  pts(A: (0, 0), B: (5, 1), C: (2, 3))
  perp("P1", "P2", ("A", "B"), "C")
  project("H", "C", "A", "B")

  line("A", "B", stroke: black)
  line("P1", "P2", stroke: blue)
  mark-right-angle("A", "H", "C", size: 0.3)

  points("A", "B", "C")
  labels("A", "B", "C",
    A: "left", B: "right", C: "above")
})
```

Figure



4.2. Parallel — `para`

Construct a parallel line through a point:

Code

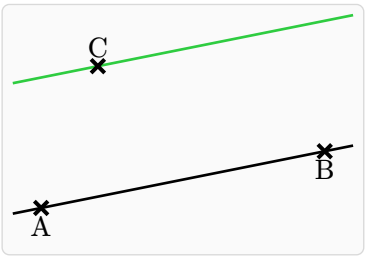
```
#canvas(length: 0.75cm, {
  import cetz.draw: *
  init()

  pts(A: (0, 0), B: (5, 1), C: (1, 2.5))
  para("P1", "P2", ("A", "B"), "C")

  set-clip(-0.5, -0.5, 5.5, 3.5)
  tkz-line("A", "B", add: (2, 2), stroke: black)
  tkz-line("P1", "P2", add: (2, 2), stroke: green)

  points("A", "B", "C")
  labels("A", "B", "C",
    A: "below", B: "below", C: "above")
})
```

Figure



4.3. Angle Bisector — `bisect`

Construct the bisector of an angle:

Code

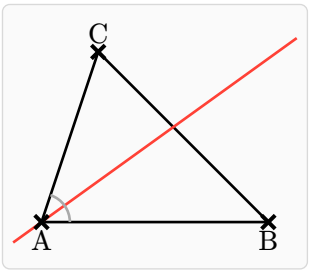
```
#canvas(length: 0.8cm, {
  import cetz.draw: *
  init()

  pts(A: (0, 0), B: (4, 0), C: (1, 3))
  bisect("D1", "D2", "C", "A", "B")

  set-clip(-0.5, -0.5, 4.5, 3.5)
  line("A", "B", "C", "A", stroke: black)
  seg("D1", "D2", stroke: red)

  angle("A", "C", "B", radius: 0.5, stroke: gray)
  points("A", "B", "C")
  labels("A", "B", "C",
    A: "below", B: "below", C: "above")
})
```

Figure



4.4. Perpendicular Bisector — `mediator`

Construct the perpendicular bisector of a segment:

Code

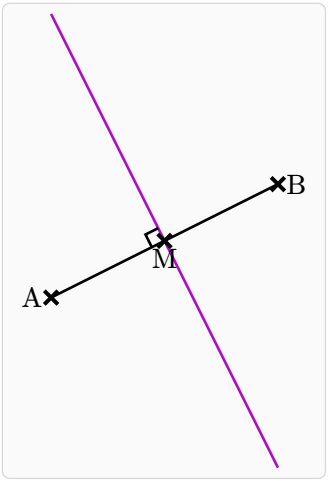
```
#canvas(length: 0.75cm, {
  import cetz.draw: *
  init()

  pts(A: (1, 1), B: (5, 3))
  mediator("M1", "M2", "A", "B")
  midpoint("M", "A", "B")

  line("A", "B", stroke: black)
  line("M1", "M2", stroke: purple)
  mark-right-angle("M1", "M", "A", size: 0.25)

  points("A", "B", "M")
  labels("A", "B", "M",
    A: "left", B: "right", M: "below")
})
```

Figure



5. Intersections

5.1. Line–Line — `ll`

Find the intersection of two lines:

Code

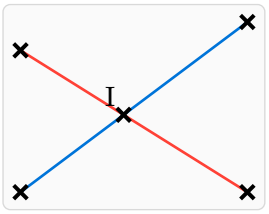
```
#canvas(length: 0.8cm, {
  import cetz.draw: *
  init()

  pts(A: (0, 0), B: (4, 3),
            C: (4, 0), D: (0, 2.5))
  ll("I", ("A", "B"), ("C", "D"))

  line("A", "B", stroke: blue)
  line("C", "D", stroke: red)

  points("A", "B", "C", "D", "I")
  labels("I", I: "above left")
})
```

Figure



5.2. Line–Circle — `lc`

Find intersections of a line with a circle:

Code

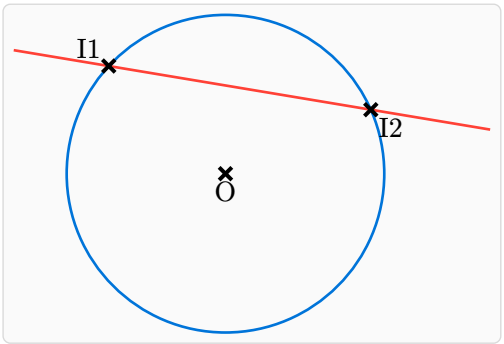
```
#canvas(length: 0.7cm, {
  import cetz.draw: *
  init()

  pts(O: (0, 0), R: (3, 0),
            A: (-2, 2), B: (4, 1))
  lc(("I1", "I2"), ("A", "B"),
    (center: "O", through: "R"))

  circle("O", "R", stroke: blue)
  set-clip(-4, -4, 5, 4)
  tkz-line("A", "B", add: (2, 2), stroke: red)

  points("O", "I1", "I2")
  labels("O", "I1", "I2",
    O: "below", I1: "above left", I2: "below right")
})
```

Figure



5.3. Circle–Circle — `cc`

Find intersections of two circles:

Code

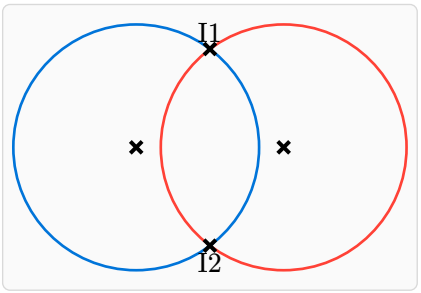
```
#canvas(length: 0.65cm, {
  import cetz.draw: *
  init()

  pts(O1: (0, 0), O2: (3, 0),
            R1: (2.5, 0), R2: (5.5, 0))
  cc(("I1", "I2"),
    (center: "O1", through: "R1"),
    (center: "O2", through: "R2"))

  circle("O1", "R1", stroke: blue)
  circle("O2", "R2", stroke: red)

  points("O1", "O2", "I1", "I2")
  labels("I1", "I2",
    I1: "above", I2: "below")
})
```

Figure



6. Triangle Centers

6.1. Basic Centers

6.1.1. Centroid — `centroid`

The intersection of medians (center of mass):

Code

```
#canvas(length: 0.8cm, {
  import cetz.draw: *
  init()

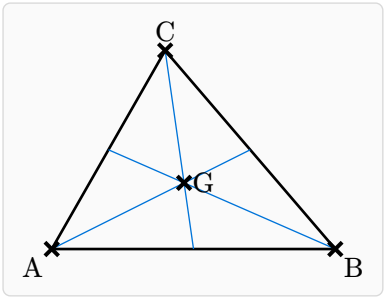
  pts(A: (0, 0), B: (5, 0), C: (2, 3.5))
  centroid("G", "A", "B", "C")

  // Draw medians
  midpoint("Ma", "B", "C")
  midpoint("Mb", "A", "C")
  midpoint("Mc", "A", "B")

  line("A", "B", "C", "A", stroke: black)
  line("A", "Ma", stroke: blue + 0.5pt)
  line("B", "Mb", stroke: blue + 0.5pt)
  line("C", "Mc", stroke: blue + 0.5pt)

  points("A", "B", "C", "G")
  labels("A", "B", "C", "G",
    A: "below left", B: "below right",
    C: "above", G: "right")
})
```

Figure



6.1.2. Circumcenter — `circumcenter`

Center of the circumscribed circle:

Code

```
#canvas(length: 0.75cm, {
  import cetz.draw: *
  init()

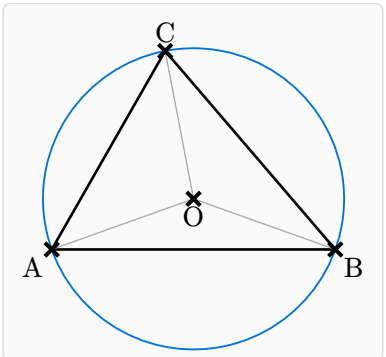
  pts(A: (0, 0), B: (5, 0), C: (2, 3.5))
  circumcenter("O", "A", "B", "C")

  line("A", "B", "C", "A", stroke: black)
  circle("O", "A", stroke: blue + 0.7pt)

  line("O", "A", stroke: gray + 0.5pt)
  line("O", "B", stroke: gray + 0.5pt)
  line("O", "C", stroke: gray + 0.5pt)

  points("A", "B", "C", "O")
  labels("A", "B", "C", "O",
    A: "below left", B: "below right",
    C: "above", O: "below")
})
```

Figure



6.1.3. Incenter — `incenter`

Center of the inscribed circle:

Code

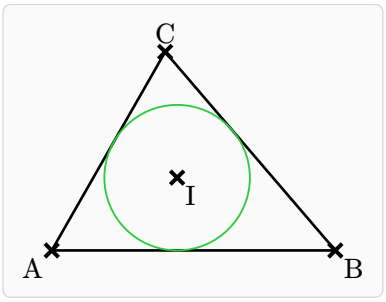
```
#canvas(length: 0.75cm, {
  import cetz.draw: *
  init()

  pts(A: (0, 0), B: (5, 0), C: (2, 3.5))
  incenter("I", "A", "B", "C")

  line("A", "B", "C", "A", stroke: black)
  incircle("A", "B", "C", stroke: green + 0.7pt)

  points("A", "B", "C", "I")
  labels("A", "B", "C", "I",
    A: "below left", B: "below right",
    C: "above", I: "below right")
})
```

Figure



6.1.4. Orthocenter — `orthocenter`

Intersection of altitudes:

Code

```
#canvas(length: 0.75cm, {
  import cetz.draw: *
  init()

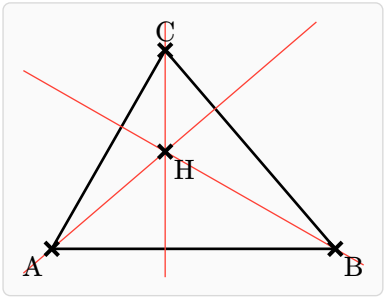
  pts(A: (0, 0), B: (5, 0), C: (2, 3.5))
  orthocenter("H", "A", "B", "C")

  // Altitudes
  perp("Ha1", "Ha2", ("B", "C"), "A")
  perp("Hb1", "Hb2", ("A", "C"), "B")
  perp("Hc1", "Hc2", ("A", "B"), "C")

  set-clip(-0.5, -0.5, 5.5, 4)
  line("A", "B", "C", "A", stroke: black)
  tkz-line("A", "Ha1", add: (2, 2), stroke: red +
0.5pt)
  tkz-line("B", "Hb1", add: (2, 2), stroke: red +
0.5pt)
  tkz-line("C", "Hc1", add: (2, 2), stroke: red +
0.5pt)

  points("A", "B", "C", "H")
  labels("A", "B", "C", "H",
    A: "below left", B: "below right",
    C: "above", H: "below right")
})
```

Figure



6.2. The Euler Line

In any non-equilateral triangle, the orthocenter H, centroid G, and circumcenter O are collinear. This line is called the **Euler line**, and remarkably, G divides HO in the ratio $2:1$.

Code

```
#canvas(length: 0.75cm, {
  import cetz.draw: *
  init()

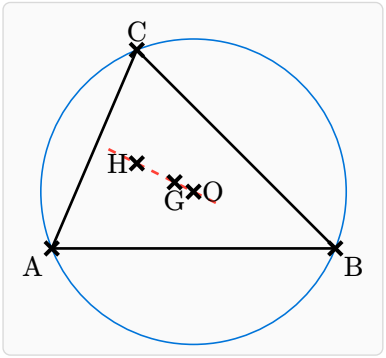
  pts(A: (0, 0), B: (5, 0), C: (1.5, 3.5))

  orthocenter("H", "A", "B", "C")
  centroid("G", "A", "B", "C")
  circumcenter("O", "A", "B", "C")

  set-clip(-0.5, -0.5, 5.5, 4)
  line("A", "B", "C", "A", stroke: black)
  line-add("H", "O", add: 0.5, stroke: (paint: red,
dash: "dashed"))
  circle("O", "A", stroke: blue + 0.6pt)

  points("A", "B", "C", "H", "G", "O")
  labels("A", "B", "C", "H", "G", "O",
    A: "below left", B: "below right", C: "above",
    H: "left", G: "below", O: "right")
})
```

Figure



6.3. Advanced Centers

`ctz-euclide` supports 10+ specialized triangle centers:

- `lemoine` — Symmedian point (Lemoine point)
- `nagel` — Nagel point
- `gergonne` — Gergonne point
- `spieker` — Spieker center (incenter of medial triangle)
- `euler` — Nine-point circle center
- `feuerbach` — Feuerbach point
- `mittenpunkt` — Mittenpunkt
- `excenter` — Excenter (specify vertex: `"a"`, `"b"`, or `"c"`)

Example with Euler (nine-point) circle:

Code

```
#canvas(length: 0.7cm, {
  import cetz.draw: *
  init()

  pts(A: (0, 0), B: (5, 0), C: (1.5, 3.5))
  euler("N", "A", "B", "C")

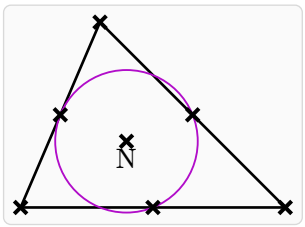
  line("A", "B", "C", "A", stroke: black)

  // Nine-point circle passes through midpoints
  midpoint("Ma", "B", "C")
  midpoint("Mb", "A", "C")
  midpoint("Mc", "A", "B")

  circle("N", "Ma", stroke: purple + 0.7pt)

  points("A", "B", "C", "N", "Ma", "Mb", "Mc")
  labels("N", N: "below")
})
```

Figure



7. Transformations

7.1. Rotation — `rotate`

Rotate a point around a center:

Code

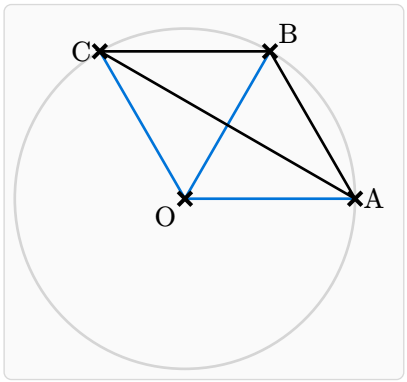
```
#canvas(length: 0.75cm, {
  import cetz.draw: *
  init()

  pts(O: (2, 2), A: (5, 2))
  rotation("B", "A", "O", 60)
  rotation("C", "A", "O", 120)

  circle("O", radius: 3, stroke: gray.lighten(50%))
  line("O", "A", stroke: blue)
  line("O", "B", stroke: blue)
  line("O", "C", stroke: blue)
  line("A", "B", "C", "A", stroke: black)

  points("O", "A", "B", "C")
  labels("O", "A", "B", "C",
    O: "below left", A: "right",
    B: "above right", C: "left")
})
```

Figure



7.2. Reflection — `reflect`

Reflect a point across a line:

Code

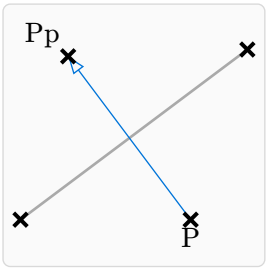
```
#canvas(length: 0.75cm, {
  import cetz.draw: *
  init()

  pts(A: (0, 0), B: (4, 3), P: (3, 0))
  reflect("Pp", "P", "A", "B")

  line("A", "B", stroke: gray)
  line("P", "Pp", stroke: blue + 0.5pt, mark: (end:
">"))

  points("A", "B", "P", "Pp")
  labels("P", "Pp", P: "below", Pp: "above left")
})
```

Figure



7.3. Homothety (Scaling) — `scale`

Scale a point from a center:

Code

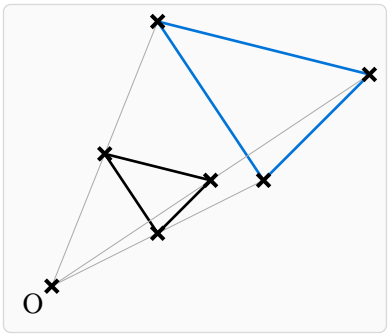
```
#canvas(length: 0.7cm, {
  import cetz.draw: *
  init()

  pts(O: (0, 0), A: (2, 1), B: (3, 2), C: (1, 2.5))
  homothety("Ap", "A", "O", 2)
  homothety("Bp", "B", "O", 2)
  homothety("Cp", "C", "O", 2)

  line("A", "B", "C", "A", stroke: black)
  line("Ap", "Bp", "Cp", "Ap", stroke: blue)
  line("O", "Ap", stroke: gray + 0.3pt)
  line("O", "Bp", stroke: gray + 0.3pt)
  line("O", "Cp", stroke: gray + 0.3pt)

  points("O", "A", "B", "C", "Ap", "Bp", "Cp")
  labels("O", O: "below left")
})
```

Figure



7.4. Projection — `project`

Project a point onto a line:

Code

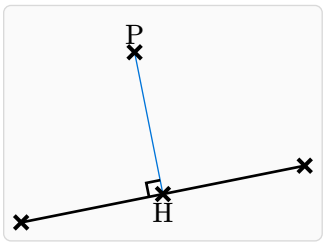
```
#canvas(length: 0.75cm, {
  import cetz.draw: *
  init()

  pts(A: (0, 0), B: (5, 1), P: (2, 3))
  project("H", "P", "A", "B")

  line("A", "B", stroke: black)
  line("P", "H", stroke: blue + 0.5pt)
  mark-right-angle("A", "H", "P", size: 0.25)

  points("A", "B", "P", "H")
  labels("P", "H", P: "above", H: "below")
})
```

Figure



8. Drawing & Styling

8.1. Points — `points`

Draw points at named locations:

```
points("A", "B", "C")
```

8.2. Labels — `labels`

Add labels to points with positioning:

```
labels("A", "B", "C",
  A: "below left",
  B: "below right",
  C: "above")
```

Positions: `"above"`, `"below"`, `"left"`, `"right"`, `"above left"`, etc.

Custom offset:

```
labels("O", O: (pos: "below", offset: (0, -0.15)))
```

8.3. Global Styling — `style`

Set default styles for points and labels:

Code

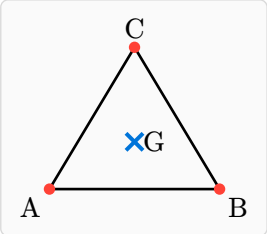
```
#canvas(length: 0.8cm, {
  import cetz.draw: *
  init()
  style(point: (shape: "dot", size: 0.1, fill: red))

  pts(A: (0, 0), B: (3, 0), C: (1.5, 2.5))
  line("A", "B", "C", "A", stroke: black)
  points("A", "B", "C")

  style(point: (shape: "cross", size: 0.15, stroke:
blue + 1.5pt))
  centroid("G", "A", "B", "C")
  points("G")

  labels("A", "B", "C", "G",
    A: "below left", B: "below right",
    C: "above", G: "right")
})
```

Figure



Point shapes: `"dot"`, `"cross"`, `"circle"`, `"square"`

8.4. Angle Marking — `angle`

Mark and label angles:

Code

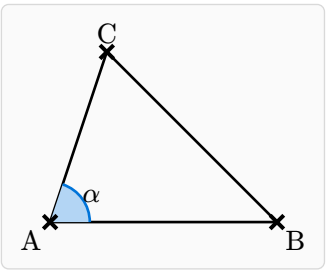
```
#canvas(length: 0.8cm, {
  import cetz.draw: *
  init()

  pts(A: (0, 0), B: (4, 0), C: (1, 3))
  line("A", "B", "C", "A", stroke: black)

  angle("A", "B", "C",
    label: $alpha$,
    radius: 0.7,
    fill: blue.lighten(70%),
    stroke: blue)

  points("A", "B", "C")
  labels("A", "B", "C",
    A: "below left", B: "below right", C: "above")
})
```

Figure



8.5. Right Angle Mark — `mark-right-angle`

Mark a right angle with a small square:

Code

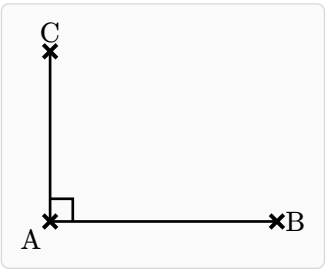
```
#canvas(length: 0.8cm, {
  import cetz.draw: *
  init()

  pts(A: (0, 0), B: (4, 0), C: (0, 3))
  line("A", "B", stroke: black)
  line("A", "C", stroke: black)

  mark-right-angle("B", "A", "C", size: 0.4)

  points("A", "B", "C")
  labels("A", "B", "C",
    A: "below left", B: "right", C: "above")
})
```

Figure



9. Circles

9.1. Circumcircle — `circumcircle`

Draw the circumscribed circle of a triangle:

```
circumcircle("A", "B", "C", stroke: blue + 1pt)
```

9.2. Incircle — `incircle`

Draw the inscribed circle of a triangle:

```
incircle("A", "B", "C", stroke: green + 1pt)
```

9.3. Circle Through Point — `circle-through`

Draw a circle with given center passing through a point:

```
circle-through("O", "A", stroke: blue)
```

9.4. Semicircle — `semicircle`

Draw a semicircle on a diameter:

```
semicircle("A", "B", stroke: blue)
```

10. Clipping

Lines that extend infinitely need to be clipped to the visible region.

10.1. Set Clip Region — `set-clip`

Define a rectangular clip boundary:

```
set-clip(xmin, ymin, xmax, ymax)
```

10.2. Draw Clipped Line — `tkz-line`

Draw a line that is automatically clipped:

```
tkz-line("A", "B", add: (2, 2), stroke: blue)
```

The `add` parameter extends the line beyond the two points before clipping.

10.3. Draw Clipped Segment — `seg`

Draw a segment with clipping:

```
seg("A", "B", stroke: red)
```

Code

```
#canvas(length: 0.7cm, {
  import cetz.draw: *
  init()

  pts(A: (0, 0), B: (2, 3))

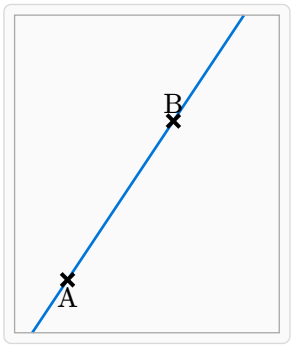
  // Set clip boundary
  set-clip(-1, -1, 4, 5)

  // Draw extended line (clipped)
  tkz-line("A", "B", add: (5, 5), stroke: blue)

  // Show clip boundary
  show-clip(stroke: gray + 0.5pt)

  points("A", "B")
  labels("A", "B", A: "below", B: "above")
})
```

Figure



11. Raw Algorithms

For direct computation without the point registry, use the `raw` dictionary:

```
// Direct centroid calculation
let center = raw.centroid((0,0,0), (3,0,0), (1.5,2.5,0))

// Distance between points
let d = raw.dist((0,0,0), (3,4,0))  // Returns 5

// Line-line intersection
let pt = raw.line-line((0,0,0), (1,1,0), (0,1,0), (1,0,0), ray: true)
```

Available raw functions:

- **Intersections**: `line-line`, `line-circle`, `circle-circle`
- **Triangle centers**: `centroid`, `circumcenter`, `incenter`, `orthocenter`, `euler-center`, `lemoine`, etc.
- **Transformations**: `rotation`, `reflection`, `translation`, `homothety`, `projection`
- **Utilities**: `midpoint`, `dist`, `angle-at-vertex`, `triangle-area`, etc.

12. Function Reference

12.1. Point Definition

- `pts(A: (x, y), ...)` — Define named points
- `midpoint(name, a, b)` — Midpoint of segment
- `linear(name, a, b, k)` — Point at ratio k along line
- `regular-polygon(names, center, first)` — Regular n-gon vertices
- `point-on-circle(name, center, radius, angle)` — Point on circle at angle
- `equilateral(name, a, b)` — Third vertex of equilateral triangle
- `golden(name, a, b)` — Golden ratio point

12.2. Line Constructions

- `perp(n1, n2, line, through)` — Perpendicular through point
- `para(n1, n2, line, through)` — Parallel through point
- `bisect(n1, n2, a, vertex, c)` — Angle bisector
- `mediator(n1, n2, a, b)` — Perpendicular bisector

12.3. Intersections

- `ll(name, line1, line2)` — Line-line intersection
- `lc(names, line, circle)` — Line-circle intersections
- `cc(names, circle1, circle2)` — Circle-circle intersections

12.4. Triangle Centers

- `centroid`, `circumcenter`, `incenter`, `orthocenter`
- `euler`, `lemoine`, `nagel`, `gergonne`, `spieker`
- `feuerbach`, `mittenpunkt`, `excenter`

12.5. Special Triangles

- `medial(na, nb, nc, a, b, c)` — Medial triangle
- `orthic(na, nb, nc, a, b, c)` — Orthic triangle
- `intouch(na, nb, nc, a, b, c)` — Intouch triangle

12.6. Transformations

- `rotation(name, source, center, angle)` — Rotation
- `reflect(name, source, line-a, line-b)` — Reflection
- `translate(name, source, vector)` — Translation
- `homothety(name, source, center, factor)` — Homothety
- `project(name, source, line-a, line-b)` — Projection

12.7. Drawing

- `points(names...)` — Draw points
- `labels(names..., positions...)` — Label points
- `style(point: (...))` — Set styling
- `angle(vertex, a, b, ...)` — Mark angle
- `mark-right-angle(a, vertex, c, ...)` — Right angle mark
- `segment(a, b, ...)` — Draw segment
- `circumcircle(a, b, c, ...)` — Circumscribed circle
- `incircle(a, b, c, ...)` — Inscribed circle

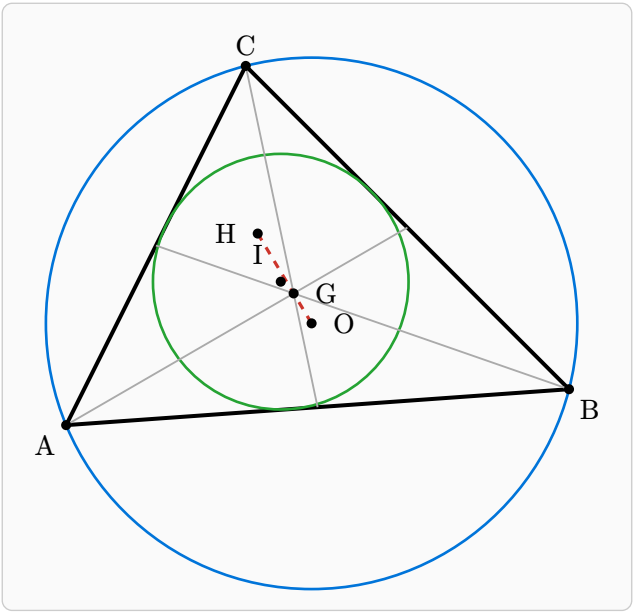
12.8. Clipping

- `set-clip(xmin, ymin, xmax, ymax)` — Set clip region
- `clear-clip()` — Clear clip region
- `tkz-line(a, b, ...)` — Draw clipped line
- `seg(a, b, ...)` — Draw clipped segment

13. Gallery Examples

The following pages showcase advanced geometric constructions using ctz-euclide. Each example demonstrates different features and techniques.

Triangle Centers




```
#canvas(length: 0.95cm, {
  import cetz.draw: *
  init()
  style(point: (shape: "dot", size: 0.07, fill: black))

  pts(A: (0, 0), B: (7, 0.5), C: (2.5, 5))
  line("A", "B", "C", "A", stroke: black + 1.5pt)

  centroid("G", "A", "B", "C")
  circumcenter("O", "A", "B", "C")
  incenter("I", "A", "B", "C")
  orthocenter("H", "A", "B", "C")

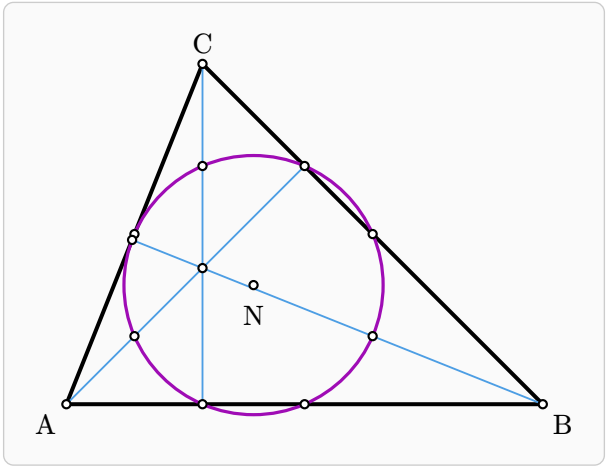
  // Euler line (H, G, O are collinear)
  line("H", "O", stroke: (
    paint: red.darken(20%),
    dash: "dashed",
    thickness: 1.2pt,
  ))

  // Circumcircle and incircle
  circle("O", "A", stroke: blue + 1pt)
  incircle("A", "B", "C", stroke: green.darken(20%) + 1pt)

  // Draw medians to centroid
  midpoint("Ma", "B", "C")
  midpoint("Mb", "A", "C")
  midpoint("Mc", "A", "B")
  line("A", "Ma", stroke: gray + 0.7pt)
  line("B", "Mb", stroke: gray + 0.7pt)
  line("C", "Mc", stroke: gray + 0.7pt)

  points("A", "B", "C", "G", "O", "I", "H")
  labels("A", "B", "C", "G", "O", "I", "H", ...)
})
```

Nine-Point Circle




```
#canvas(length: 0.9cm, {
  import cetz.draw: *
  init()
  style(point: (shape: "circle", size: 0.06, stroke: black + 0.8pt, fill: white))

  pts(A: (0, 0), B: (7, 0), C: (2, 5))
  line("A", "B", "C", "A", stroke: black + 1.5pt)

  // Midpoints of sides
  midpoint("Ma", "B", "C")
  midpoint("Mb", "A", "C")
  midpoint("Mc", "A", "B")

  // Feet of altitudes
  project("Ha", "A", "B", "C")
  project("Hb", "B", "A", "C")
  project("Hc", "C", "A", "B")

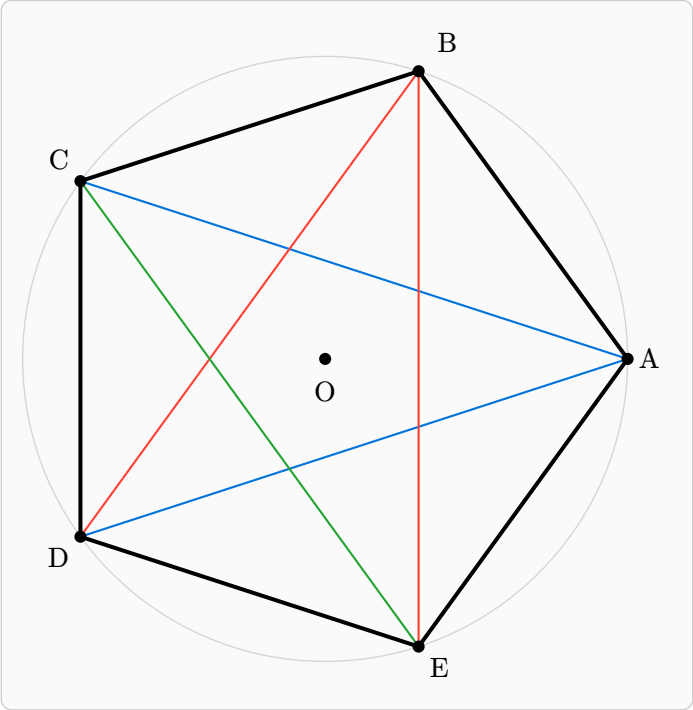
  // Orthocenter and midpoints to vertices
  orthocenter("H", "A", "B", "C")
  midpoint("MHa", "H", "A")
  midpoint("MHb", "H", "B")
  midpoint("MHc", "H", "C")

  // Nine-point circle passes through all 9 points
  euler("N", "A", "B", "C")
  circle("N", "Ma", stroke: purple.darken(10%) + 1.2pt)

  // Draw altitudes
  line("A", "Ha", stroke: blue.lighten(30%) + 0.7pt)
  line("B", "Hb", stroke: blue.lighten(30%) + 0.7pt)
  line("C", "Hc", stroke: blue.lighten(30%) + 0.7pt)

  points("A", "B", "C", "N", "Ma", "Mb", "Mc", "Ha", "Hb", "Hc", "MHa", "MHb", "MHc", "H")
  labels("A", "B", "C", "N", ...)
})
```

Regular Pentagon




```
#canvas(length: 1cm, {
  import cetz.draw: *
  init()
  style(point: (shape: "dot", size: 0.08, fill: black))

  pts(O: (0, 0), V1: (4, 0))
  regular-polygon(("A", "B", "C", "D", "E"), "O", "V1")

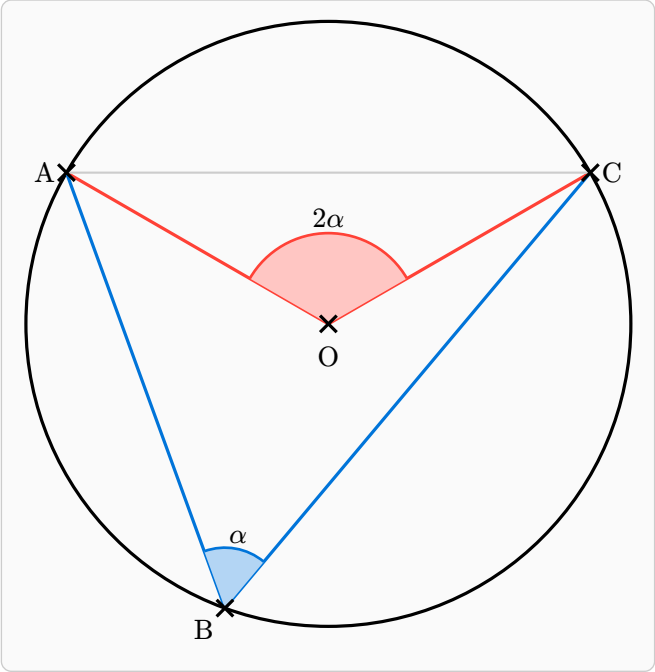
  // Pentagon
  line("A", "B", "C", "D", "E", "A", stroke: black + 1.5pt)

  // All diagonals
  line("A", "C", stroke: blue + 0.8pt)
  line("A", "D", stroke: blue + 0.8pt)
  line("B", "D", stroke: red + 0.8pt)
  line("B", "E", stroke: red + 0.8pt)
  line("C", "E", stroke: green.darken(20%) + 0.8pt)

  // Center
  circle("O", radius: 4, stroke: gray.lighten(50%) + 0.5pt)

  points("A", "B", "C", "D", "E", "O")
  labels("A", "B", "C", "D", "E", "O", ...)
})
```

Inscribed Angle Theorem




```
#canvas(length: 1cm, {
  import cetz.draw: *
  init()
  style(point: (shape: "cross", size: 0.11, stroke: black + 1.2pt))

  pts(O: (0, 0), R: (4, 0))
  circle("O", radius: 4, stroke: black + 1.2pt)

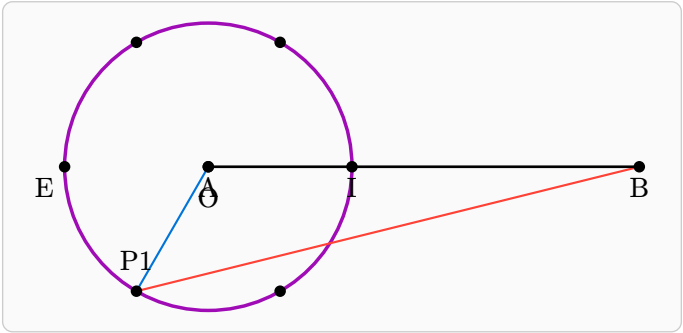
  // Place points on circle
  rotation("A", "R", "O", 150)
  rotation("C", "R", "O", 30)
  rotation("B", "R", "O", 250)

  // Inscribed angle at B
  line("A", "B", stroke: blue + 1.2pt)
  line("B", "C", stroke: blue + 1.2pt)
  angle("B", "A", "C", label: $alpha$, radius: 0.8,
    fill: blue.lighten(70%), stroke: blue)

  // Central angle at O (twice the inscribed angle)
  line("O", "A", stroke: red + 1.2pt)
  line("O", "C", stroke: red + 1.2pt)
  angle("O", "A", "C", label: $2alpha$, radius: 1.2,
    fill: red.lighten(70%), stroke: red)

  line("A", "C", stroke: gray.lighten(40%) + 0.8pt)
  points("O", "A", "B", "C")
  labels("O", "A", "B", "C", ...)
})
```

Apollonius Circle




```
#canvas(length: 0.95cm, {
  import cetz.draw: *
  init()
  style(point: (shape: "dot", size: 0.08, fill: black))

  pts(A: (-3, 0), B: (3, 0))

  // Apollonius circle: locus of points P where PA/PB = k
  let k = 2

  // External and internal division points
  pts(E: (-5, 0), I: (-1, 0))

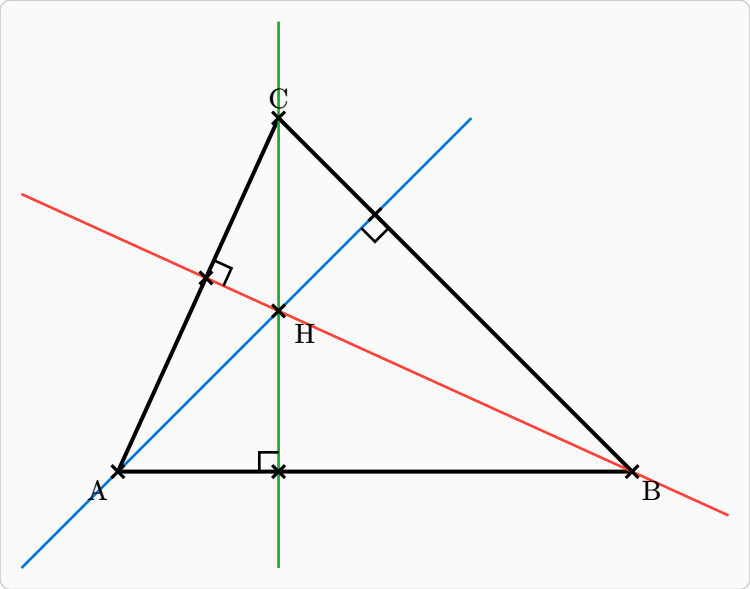
  // Center is midpoint of E and I
  midpoint("O", "E", "I")
  circle("O", "E", stroke: purple.darken(10%) + 1.3pt)

  // Show some points on the circle
  rotation("P1", "E", "O", 60)
  rotation("P2", "E", "O", 120)
  rotation("P3", "E", "O", -60)
  rotation("P4", "E", "O", -120)

  // For P1: PA/PB = k = 2
  line("P1", "A", stroke: blue + 0.8pt)
  line("P1", "B", stroke: red + 0.8pt)
  line("A", "B", stroke: black + 1pt)

  points("A", "B", "O", "E", "I", "P1", "P2", "P3", "P4")
  labels("A", "B", "E", "I", "P1", "O", ...)
})
```

Orthocenter and Altitudes




```
#canvas(length: 0.85cm, {
  import cetz.draw: *
  init()
  style(point: (shape: "cross", size: 0.1, stroke: black + 1.2pt))

  pts(A: (0, 0), B: (8, 0), C: (2.5, 5.5))
  set-clip(-1.5, -1.5, 9.5, 7)

  // Triangle
  line("A", "B", "C", "A", stroke: black + 1.5pt)

  // Extended altitudes (automatically clipped)
  perp("Ha1", "Ha2", ("B", "C"), "A")
  perp("Hb1", "Hb2", ("A", "C"), "B")
  perp("Hc1", "Hc2", ("A", "B"), "C")

  tkz-line("A", "Ha1", add: (1, 1.5), stroke: blue + 1pt)
  tkz-line("B", "Hb1", add: (1, 1.5), stroke: red + 1pt)
  tkz-line("C", "Hc1", add: (1, 2.5), stroke: green.darken(20%) + 1pt)

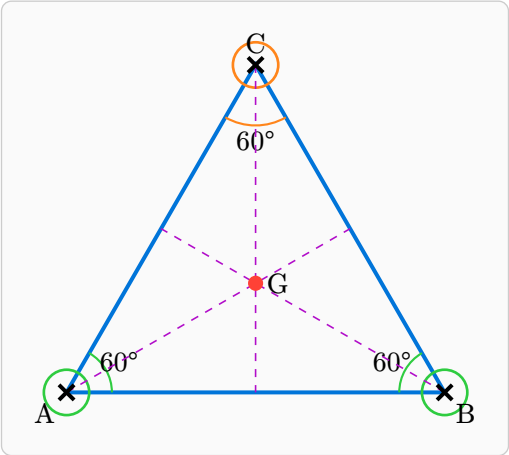
  // Orthocenter (intersection of altitudes)
  orthocenter("H", "A", "B", "C")

  // Feet of altitudes
  project("Ha", "A", "B", "C")
  project("Hb", "B", "A", "C")
  project("Hc", "C", "A", "B")

  mark-right-angle("A", "Ha", "B", size: 0.3)
  mark-right-angle("B", "Hb", "C", size: 0.3)
  mark-right-angle("C", "Hc", "A", size: 0.3)

  points("A", "B", "C", "H", "Ha", "Hb", "Hc")
  labels("A", "B", "C", "H", ...)
})
```

Equilateral Triangle Construction




```
#canvas({
  import cetz.draw: *
  init()
  style(point: (shape: "cross", size: 0.1, stroke: black + 1.5pt))

  // Base of equilateral triangle
  pts("A", (0, 0), "B", (5, 0))
  equilateral("C", "A", "B")

  // Draw triangle
  line("A", "B", "C", "A", stroke: blue + 1.5pt)

  // Mark 60° angles
  angle("A", "B", "C", label: $60°$, radius: 0.6, stroke: green + 0.8pt)
  angle("B", "C", "A", label: $60°$, radius: 0.6, stroke: green + 0.8pt)
  angle("C", "A", "B", label: $60°$, radius: 0.8, stroke: orange + 0.8pt)

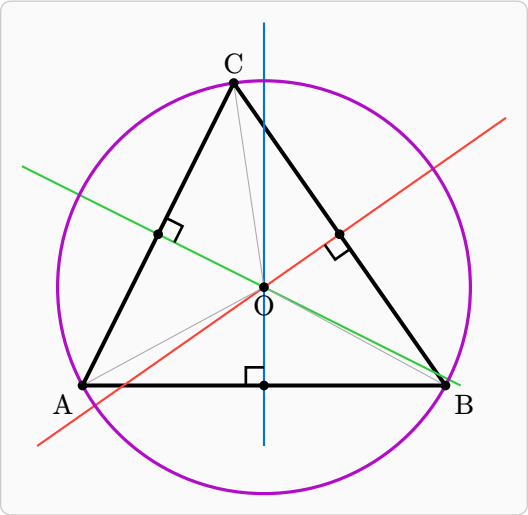
  // Draw circles at vertices
  circle("A", radius: 0.3, stroke: green + 1pt)
  circle("B", radius: 0.3, stroke: green + 1pt)
  circle("C", radius: 0.3, stroke: orange + 1pt)

  // In equilateral triangle, all centers coincide
  centroid("G", "A", "B", "C")

  // Draw medians/altitudes
  midpoint("Ma", "B", "C")
  midpoint("Mb", "A", "C")
  midpoint("Mc", "A", "B")
  line("A", "Ma", stroke: (paint: purple, thickness: 0.6pt, dash: "dashed"))
  line("B", "Mb", stroke: (paint: purple, thickness: 0.6pt, dash: "dashed"))
  line("C", "Mc", stroke: (paint: purple, thickness: 0.6pt, dash: "dashed"))

  points("A", "B", "C", "G")
  labels("A", "B", "C", "G", ...)
})
```

Perpendicular Bisectors and Circumcircle




```
#canvas({
  import cetz.draw: *
  init()
  style(point: (shape: "dot", size: 0.08, fill: black))

  // Define triangle
  pts("A", (0, 0), "B", (6, 0), "C", (2.5, 5))
  set-clip(-1, -1, 7, 6)

  // Calculate midpoints
  midpoint("Mab", "A", "B")
  midpoint("Mbc", "B", "C")
  midpoint("Mca", "C", "A")

  // Circumcenter is intersection of perpendicular bisectors
  circumcenter("O", "A", "B", "C")

  // Create perpendicular bisector lines
  mediator("Pab1", "Pab2", "A", "B")
  mediator("Pbc1", "Pbc2", "B", "C")
  mediator("Pca1", "Pca2", "C", "A")

  // Draw triangle
  line("A", "B", "C", "A", stroke: black + 1.5pt)

  // Draw perpendicular bisectors (clipped)
  seg("Pab1", "Pab2", stroke: blue + 0.8pt)
  seg("Pbc1", "Pbc2", stroke: red + 0.8pt)
  seg("Pca1", "Pca2", stroke: green + 0.8pt)

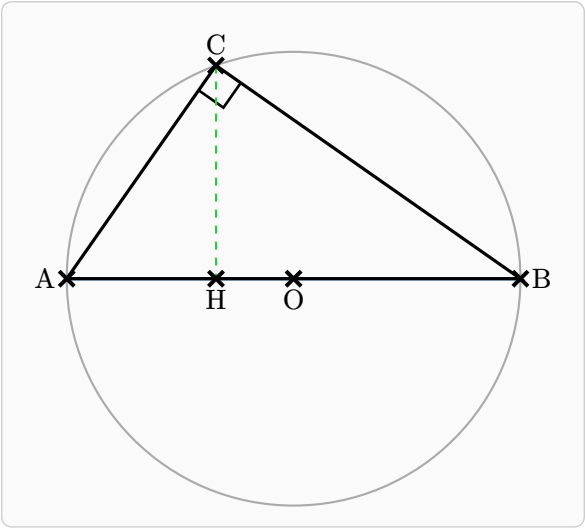
  // Draw circumcircle
  circumcircle("A", "B", "C", stroke: purple + 1.2pt)

  // Radii (equal length)
  line("O", "A", stroke: gray + 0.4pt)
  line("O", "B", stroke: gray + 0.4pt)
  line("O", "C", stroke: gray + 0.4pt)

  mark-right-angle("A", "Mab", "O", size: 0.3)
  mark-right-angle("B", "Mbc", "O", size: 0.3)
  mark-right-angle("C", "Mca", "O", size: 0.3)

  points("A", "B", "C", "O", "Mab", "Mbc", "Mca")
  labels("A", "B", "C", "O", ...)
})
```

Thales' Theorem




```
#canvas({
  import cetz.draw: *
  init()
  style(point: (shape: "cross", size: 0.1, stroke: black + 1.5pt))

  // Circle with diameter AB
  pts("O", (0, 0), "A", (-3, 0), "B", (3, 0))

  // Point C on circle
  point-on-circle("C", "O", 3, 110)

  // Draw circle and diameter
  circle("O", radius: 3, stroke: gray + 0.8pt)
  line("A", "B", stroke: blue + 1.2pt)

  // Draw triangle ACB
  // By Thales' theorem: angle ACB = 90° (inscribed in semicircle)
  line("A", "C", "B", "A", stroke: black + 1.2pt)

  // Mark the right angle at C
  mark-right-angle("A", "C", "B", size: 0.4)

  // Draw altitude from C to AB
  project("H", "C", "A", "B")
  line("C", "H", stroke: (paint: green, thickness: 0.8pt, dash: "dashed"))

  points("A", "B", "C", "O", "H")
  labels("A", "B", "C", "O", "H", ...)
})
```